TRƯỜNG ĐẠI HỌC ĐÔNG Á
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO

HỌC PHẦN: DEVOPS

ĐỀ TÀI:

THỰC HÀNH XÂY DỰNG MÔI TRƯỜNG DEVOPS VỚI WSL UBUNTU, SSH, DOCKER VÀ JENKINS CI/CD

GVHD: TS. Vũ Anh Tuấn

Sinh viên thực hiện:

Nguyễn Danh Tài — 102374 — IT23M

Đắk Lắk, tháng 10 năm 2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU 4
1.1. Bối cảnh và lý do thực hiện 4
1.2. Mục tiêu 4
1.3. Phạm vi và nội dung thực hiện 4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT 6
2.1. DevOps và CI/CD 6
2.2. WSL - Windows Subsystem for Linux 6
2.3. SSH - Secure Shell 6
2.4. Docker và container 6
2.5. Jenkins 7
CHƯƠNG 3. MÔI TRƯỜNG THỰC HIỆN 8
CHƯƠNG 4. TRIỂN KHAI THỰC HIỆN 9
4.1. Tạo docker-compose.yml và chạy lần đầu 9
4.2. Kiểm tra container, push image và build lại 10
4.3. Hoàn tất build lại, push bản mới và kết quả 11
4.4. Viết Jenkins cho dự án 13
CHƯƠNG 5. ĐÁNH GIÁ VÀ KẾT LUẬN 17
5.1. Kết quả đạt được 17
5.2. Một số lưu ý và cách xử lý 17
5.3. Hướng phát triển 18
5.4. Kết luận 18
TÀI LIỆU THAM KHẢO 19

DANH SÁCH HÌNH

Hình 4.1. Tạo docker-compose.yml và chạy docker compose up -d lần đầu.....10

Hình 4.2. Kiểm tra container, push image và build lại bằng docker compose up -d --build.....11

Hình 4.3. Hoàn tất build lại và push bản mới lên Docker Hub.....11

Hình 4.4. Giao diện web tại localhost:8000 sau khi build lại.....12

Hình 4.5. Docker Desktop — danh sách container đang chạy.....12

Hình 4.6. Kết quả Build #4 của job student-cicd trên Jenkins.....15

CHƯƠNG 1. GIỚI THIỆU

1.1. Bối cảnh và lý do thực hiện

Trong phát triển phần mềm hiện đại, việc đưa một thay đổi mã nguồn từ máy lập trình viên đến môi trường chạy thật cần diễn ra nhanh, lặp lại được và ít sai sót. Khi khâu phát triển và khâu vận hành làm việc tách rời, mỗi lần phát hành lại là một lần thao tác thủ công, rất dễ phát sinh lỗi cấu hình và sự cố. Mô hình DevOps ra đời nhằm rút ngắn khoảng cách giữa hai nhóm này thông qua văn hóa hợp tác, tự động hóa quy trình và đo lường liên tục.

Để áp dụng DevOps, người kỹ sư cần làm chủ một chuỗi công cụ: môi trường Linux để làm việc với dòng lệnh, giao thức SSH để điều khiển máy từ xa, container (Docker) để đóng gói ứng dụng cùng phụ thuộc vào một image và chạy đồng nhất ở mọi nơi, kho image để phân phối bản build, và máy chủ tự động hóa (Jenkins) để xây dựng quy trình CI/CD — mỗi lần có commit mới, hệ thống sẽ tự build, tự kiểm thử và tự triển khai. Báo cáo này ghi lại quá trình thực hành trên một máy Windows cá nhân với hai nhóm công việc chính: thao tác Docker với docker-compose và xây pipeline Jenkins cho dự án quản lý sinh viên.

1.2. Mục tiêu

Sau khi hoàn thành thực hành, người thực hiện đạt được các mục tiêu sau:

1. Sử dụng Docker Desktop và viết file docker-compose.yml để mô tả và chạy một ứng dụng nhiều service bằng Docker Compose.
2. Thực hiện các thao tác image cơ bản: build, quản lý tag, kiểm tra container đang chạy và đẩy image lên Docker Hub.
3. Cài đặt Jenkins chạy trong container và xây dựng job kiểu Pipeline khai báo (Declarative Pipeline).
4. Xây dựng pipeline CI/CD hoàn chỉnh cho dự án Student Management System theo chuỗi: BUILD → TEST → PUSH → PULL → DEPLOY.
5. Áp dụng smoke test tự động trước khi push image, đẩy image đa kiến trúc (linux/amd64 và linux/arm64), và triển khai bằng đúng image đã lấy từ registry — không build lại lúc deploy.

1.3. Phạm vi và nội dung thực hiện

Phạm vi thực hành gồm hai nhóm công việc: (1) bài thực hành Docker với một ứng dụng web nhỏ trong thư mục docker-lab, image được đẩy lên Docker Hub; (2) xây pipeline Jenkins cho dự án Student Management System trên nhánh DanhTai của kho

mã nguồn GitHub. Kết quả được minh chứng bằng ảnh chụp màn hình và console output của Jenkins.

Lưu ý: phần thực hành cài đặt Ubuntu trên WSL và kết nối SSH từ Ubuntu vào Windows đã được tiến hành nhưng gặp lỗi trong quá trình cấu hình, không đạt được kết quả mong muốn; theo hướng dẫn, phần này được lược bỏ khỏi báo cáo. Vì vậy nội dung thực hành còn lại tập trung vào Docker (mục 4.1–4.3) và Jenkins (mục 4.4).

Toàn bộ kết quả thực hành được trình bày tuần tự tại Chương 4, gồm bốn nội dung: tạo docker-compose.yml và chạy lần đầu; kiểm tra container, push image và build lại; hoàn tất push bản mới và kiểm tra kết quả; viết Jenkins cho dự án.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. DevOps và CI/CD

DevOps là tập hợp các thực hành, công cụ và văn hóa nhằm tự động hóa và tích hợp quy trình giữa phát triển phần mềm và vận hành hạ tầng, với mục tiêu phát hành nhanh hơn, ổn định hơn và dễ theo dõi hơn.

CI (Continuous Integration — Tích hợp liên tục) là thực hành mà lập trình viên thường xuyên hợp nhất mã nguồn vào nhánh chung; mỗi lần hợp nhất, hệ thống tự động build và chạy kiểm thử để phát hiện lỗi sớm. CD (Continuous Delivery/Deployment — Chuyển giao/Triển khai liên tục) mở rộng CI bằng việc tự động đưa bản build đạt yêu cầu đến môi trường kiểm thử hoặc môi trường thực. Một pipeline CI/CD điển hình gồm các giai đoạn: lấy mã nguồn (Checkout), biên dịch/đóng gói (Build), kiểm thử (Test), đóng gói container (Package) và triển khai (Deploy).

2.2. WSL - Windows Subsystem for Linux

WSL cho phép chạy môi trường Linux thật ngay trên Windows mà không cần cài song song hai hệ điều hành. Ở phiên bản WSL2, nhân Linux được Microsoft đóng gói trong một máy ảo siêu nhỏ, nên tương thích tốt với các công cụ như Docker. Các bản phân phối Linux được cài từ Microsoft Store hoặc bằng lệnh wsl --install; ổ đĩa Windows được gắn tự động vào Linux dưới thư mục /mnt/c, /mnt/d..., giúp hai hệ điều hành chia sẻ tệp dễ dàng. Thực hành cài đặt Ubuntu trên WSL đã được tiến hành nhưng gặp lỗi, chưa đạt kết quả (đã nêu tại mục 1.3) nên không có hình ảnh minh chứng trong báo cáo.

2.3. SSH - Secure Shell

SSH là giao thức mạng mã hóa dùng để đăng nhập và thực thi lệnh trên máy từ xa một cách an toàn, hoạt động theo mô hình client-server: máy chủ chạy dịch vụ sshd (mặc định cổng 22), máy khách dùng lệnh ssh user@host để kết nối; việc xác thực có thể bằng mật khẩu hoặc bằng cặp khóa công khai/bí mật. Trong môi trường DevOps, SSH là cách phổ biến để kỹ sư điều khiển các máy chủ từ xa chỉ bằng dòng lệnh. Tương tự mục WSL, phần thực hành SSH từ Ubuntu vào Windows gặp lỗi nên được lược bỏ theo lưu ý tại mục 1.3.

2.4. Docker và container

Docker là nền tảng đóng gói ứng dụng cùng toàn bộ thư viện và cấu hình cần thiết vào một image; từ image có thể khởi chạy nhiều container — các tiến trình được cách ly nhưng dùng chung nhân hệ điều hành host nên rất nhẹ và khởi động nhanh. Nhờ đó,

ứng dụng chạy giống nhau trên máy lập trình viên, máy kiểm thử và máy chủ thật. Image được quản lý qua các tag (ví dụ latest, hay tag theo commit) và lưu tại các registry như Docker Hub hay GitHub Container Registry (GHCR); Alpine Linux thường được dùng làm image nền vì cực kỳ nhỏ gọn.

Docker Compose cho phép mô tả toàn bộ ứng dụng nhiều service trong một file YAML duy nhất docker-compose.yml: service nào dùng image nào, cổng nào được mở ra ngoài, volume nào được lưu trữ. Với file này, chỉ cần một lệnh docker compose up -d là cả bộ dịch vụ được tạo và chạy; docker compose ps để kiểm tra trạng thái; docker compose down để dọn dẹp. Cách làm này giúp hạ tầng được mã hóa (infrastructure as code): ai có file compose đều tái lập được cùng một môi trường.

2.5. Jenkins

Jenkins là máy chủ tự động hóa mã nguồn mở, được dùng phổ biến để xây dựng pipeline CI/CD. Jenkins có hệ sinh thái plugin phong phú (Git, Docker, Pipeline, Credentials...), hỗ trợ định nghĩa quy trình bằng mã thông qua tệp Jenkinsfile theo kiểu Declarative hoặc Scripted. Mỗi lần chạy pipeline được gọi là một build và được đánh số tăng dần (#1, #2, ...). Quy trình có thể được kích hoạt bằng webhook từ GitHub hoặc bằng Poll SCM — Jenkins định kỳ dò xem kho mã nguồn có commit mới hay không. Thông tin nhạy cảm như token registry được lưu bằng tính năng Credentials của Jenkins và chỉ được giải nén ra môi trường trong lúc chạy.

CHƯƠNG 3. MÔI TRƯỜNG THỰC HIỆN

Bảng 3.1 tóm tắt các thành phần môi trường được ghi nhận trong quá trình thực hành.

Bảng 3.1. Thông tin môi trường thực hiện

Thành phần	Giá trị	Nguồn ghi nhận
Hệ điều hành chủ	Windows 10 (Microsoft NT 10.0.19045), tên máy DESKTOP-DG1FADB	Thông tin hệ thống Windows
WSL	WSL2 — Ubuntu đã cài đặt (trạng thái Stopped); thực hành gặp lỗi nên lược (mục 1.3)	Lệnh wsl -l -v
Docker Desktop	Engine 29.7.2 (linux/amd64); Docker Compose v5.5.0	Hình 4.1
Jenkins	Chạy trong container jenkins, giao diện http://localhost:8090, phiên bản 2.541.3	Hình 4.6
Kho mã nguồn	GitHub: https://github.com/phuoctoan102334/DevOPS-26-13.git — nhánh DanhTai	Hình 4.6
Kho image	Docker Hub: danhtai123/my-webapp · GHCR: ghcr.io/technology-assassin/student-management-*	Hình 4.3, Console Output
Dự án ứng dụng	Student Management System: backend Spring Boot 3.4.1 (Java 21), frontend nginx, database SQL Server 2022	docker-compose.yml trong repo
Cổng ứng dụng	3000 (frontend), 8080 (backend), 1433 (database); smoke test: 3999 / 55999 / 15333	Hình 4.4, Hình 4.6

CHƯƠNG 4. TRIỂN KHAI THỰC HIỆN

4.1. Tạo docker-compose.yml và chạy lần đầu

A. Chuẩn bị

Thư mục thực hành là D:\docker-lab, gồm ứng dụng web nhỏ viết bằng Python (tệp app.py) và tệp Dockerfile để đóng gói ứng dụng thành image. File docker-compose.yml được tạo với nội dung tóm tắt như sau, image lấy từ Docker Hub:

```
services:
  my-webapp:
    image: danhtai123/my-webapp:latest
    ports:
      - "8000:8000"
```

B. Chạy và kiểm tra

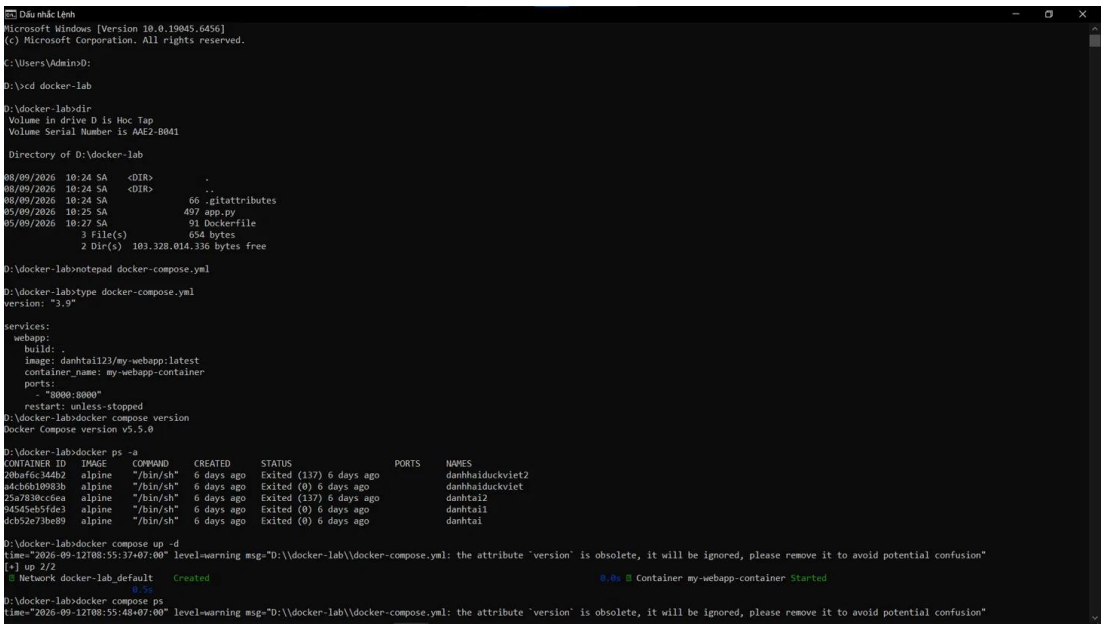
Sau khi kiểm tra phiên bản Docker Compose, lệnh docker compose up -d được dùng để chạy. Kết quả cho thấy Docker tự tạo network, tạo và khởi động container; docker compose ps xác nhận service đang ở trạng thái chạy. Bảng 4.1 giải thích các tham số của lệnh chạy; Bảng 4.2 tổng hợp các lệnh đã dùng trong mục này.

Bảng 4.1. Giải thích tham số của lệnh docker compose up

Tham số	Ý nghĩa
-d (detach)	Chạy container ở chế độ nền, giải phóng cửa sổ dòng lệnh
--build	Build lại image theo cấu hình hiện tại trước khi khởi động container
tên service (vd: my-webapp)	Chỉ định chạy một service; mặc định là chạy toàn bộ service trong file compose

Bảng 4.2. Các lệnh sử dụng trong mục 4.1

Lệnh	Ý nghĩa
docker compose version	Kiểm tra phiên bản Docker Compose đã cài (v5.5.0)
docker compose up -d	Tạo network, container và chạy nền toàn bộ service mô tả trong file compose
docker compose ps	Liệt kê service đang chạy kèm trạng thái và cổng
docker compose down	Dừng và xóa container/network của compose (giữ lại dữ liệu volume)



Hình 4.1. Tạo docker-compose.yml và chạy docker compose up -d lần đầu

Kết luận: đã tạo được file docker-compose.yml mô tả ứng dụng và chạy thành công container bằng Docker Compose; service ở trạng thái chạy và cổng đã được mở.

4.2. Kiểm tra container, push image và build lại

A. Kiểm tra container và thử push image

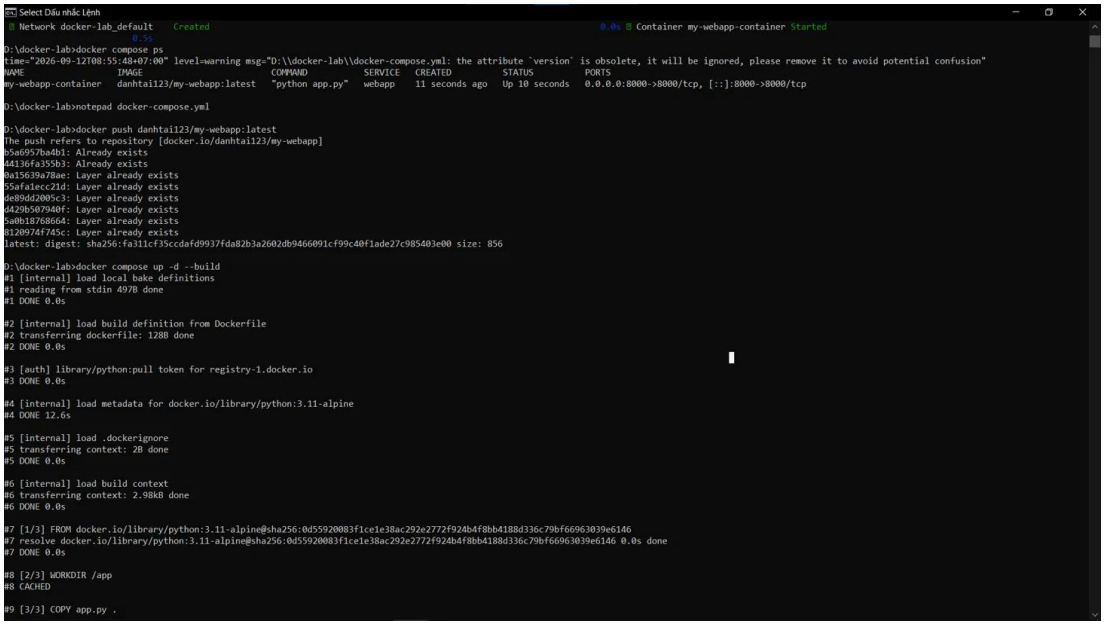
Trước khi thay đổi, docker compose ps được chạy lại để xác nhận container đang hoạt động. Tiếp theo, file compose được mở để chỉnh sửa cấu hình, sau đó thử đẩy image lên Docker Hub bằng lệnh docker push danhtai123/my-webapp:latest. Docker báo rằng tag này đã tồn tại trên registry, tức là bản image cũ vẫn còn — muốn có bản mới thì phải build lại image rồi đẩy lần nữa.

B. Build lại image theo cấu hình mới

Quyết định thực hiện là chạy docker compose up -d --build để build image theo cấu hình hiện tại rồi khởi động lại container; docker images được dùng để kiểm tra danh sách image tại máy. Bảng 4.3 tóm tắt các lệnh của bước này.

Bảng 4.3. Các lệnh sử dụng trong mục 4.2

Lệnh	Ý nghĩa
docker compose ps	Xác nhận container đang chạy (Up) và cổng đã mở
docker push danhtai123/my-webapp:latest	Đẩy image lên Docker Hub; nếu tag đã tồn tại thì cần build bản mới để có digest mới
docker compose up -d --build	Build lại image rồi khởi động lại container trong chế độ nền
docker images	Xem danh sách image tại máy kèm kích thước



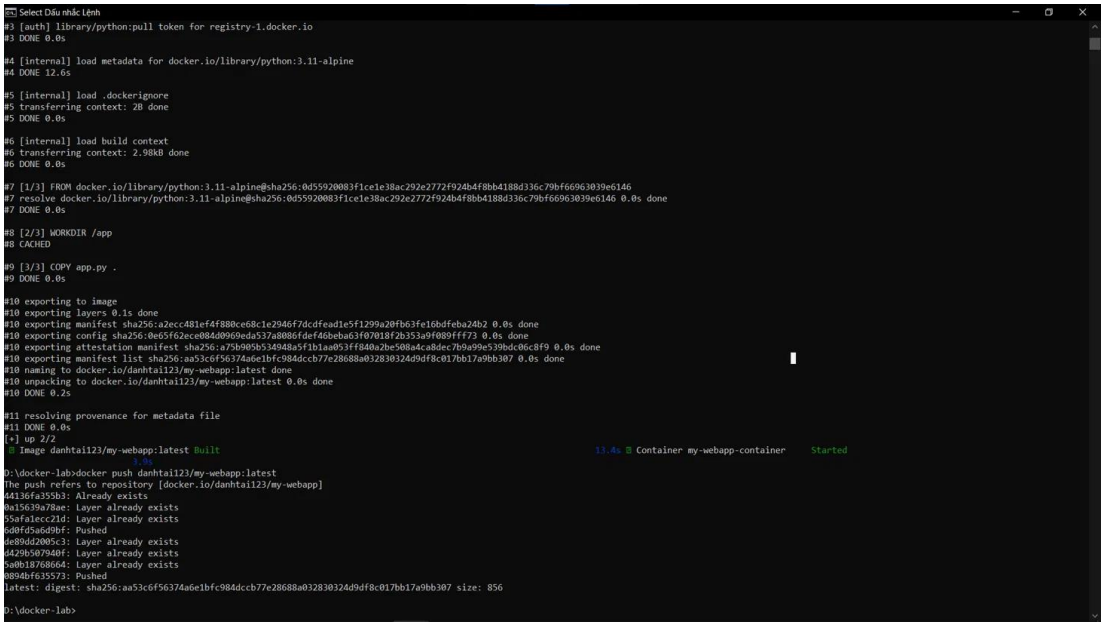
Hình 4.2. Kiểm tra container, push image và build lại bằng docker compose up -d --build

Kết luận: đã kiểm tra được trạng thái container, nhận diện được vấn đề tag image đã tồn tại trên registry và tiến hành build lại image trước khi push.

4.3. Hoàn tất build lại, push bản mới và kết quả

A. Push bản mới lên Docker Hub

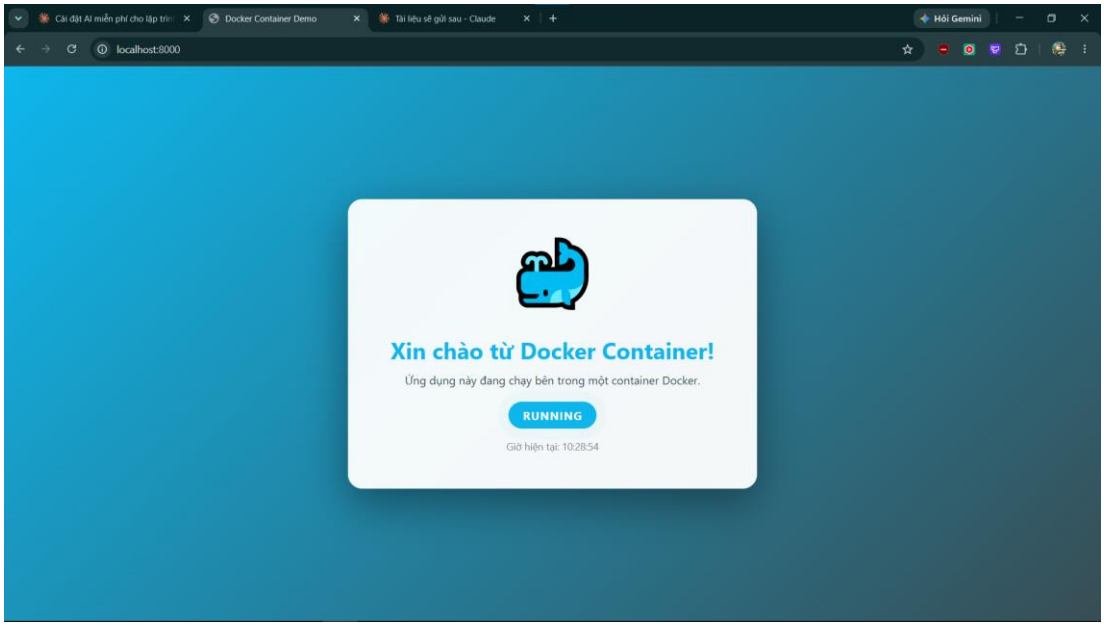
Lệnh build hoàn tất, container được Docker khởi động lại tự động. Lệnh push được chạy thêm lần nữa và lần này các layer của image được đẩy lên Docker Hub thành công, một digest mới được ghi nhận — đồng nghĩa bản build mới đã có trên registry.



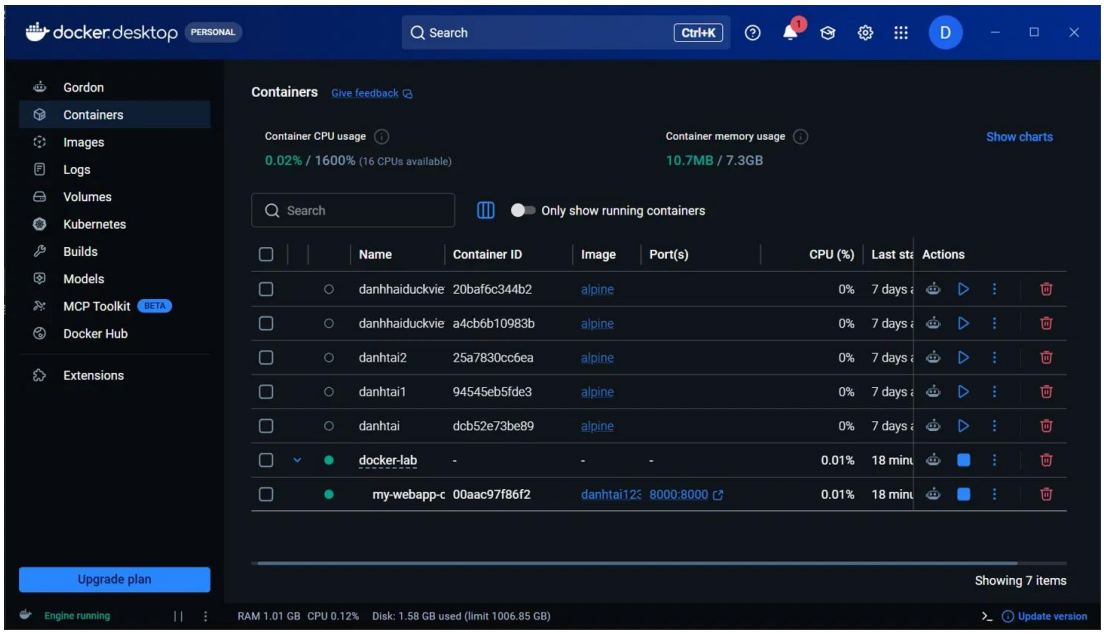
Hình 4.3. Hoàn tất build lại và push bản mới lên Docker Hub

B. Kiểm tra kết quả

Kiểm tra kết quả bằng trình duyệt: tại địa chỉ http://localhost:8000 hiển thị trang web "Xin chào từ Docker Container!" — chứng minh container đang phục vụ ứng dụng bình thường (Hình 4.4). Trên Docker Desktop, danh sách container hiển thị container của dự án docker-lab với trạng thái Running và ánh xạ cổng 8000:8000 (Hình 4.5). Bảng 4.4 tóm tắt kết quả.



Hình 4.4. Giao diện web tại localhost:8000 sau khi build lại



Hình 4.5. Docker Desktop — danh sách container đang chạy

Bảng 4.4. Tóm tắt kết quả mục 4.3

Hạng mục	Kết quả
Build lại image	Hoàn tất; container được khởi động lại tự động sau build

Push lên Docker Hub	Thành công — các layer được đẩy, digest mới được ghi nhận
Truy cập ứng dụng	http://localhost:8000 hiển thị "Xin chào từ Docker Container!"
Trạng thái container	Chạy ổn định trên Docker Desktop, cổng 8000:8000

Kết luận: chu trình build → chạy → push image lên Docker Hub đã hoạt động trọn vẹn và ứng dụng phản hồi đúng tại cổng 8000.

4.4. Viết Jenkins cho dự án

A. Mô hình CI của dự án

Dự án Student Management System gồm ba service: backend Spring Boot, frontend nginx và database SQL Server, được mô tả bằng docker-compose.yml. Pipeline Jenkins được thiết kế theo triết lý: build một lần, kiểm thử bằng đúng image vừa build, chỉ push lên registry khi kiểm thử đạt, và triển khai bằng image đã kéo về từ registry — tuyệt đối không build lại lúc deploy. Toàn bộ được định nghĩa trong tệp Jenkinsfile đặt tại thư mục gốc của kho mã nguồn.

Job Jenkins có tên student-cicd được tạo theo kiểu Pipeline from SCM: trỏ đến kho https://github.com/phuoctoan102334/DevOPS-26-13.git, nhánh */DanhTai, tệp kịch bản Jenkinsfile. Vì máy thực hành không có địa chỉ IP công khai nên không nhận webhook từ GitHub được; thay vào đó Jenkinsfile khai báo Poll SCM mỗi 2 phút một lần (H/2 * * * *). Token đăng nhập kho image GHCR được lưu trong Jenkins Credentials với ID ghcr-credentials và chỉ được truyền vào docker login qua --password-stdin để không lộ trong log. Bảng 4.5 mô tả vai trò của từng giai đoạn.

Bảng 4.5. Vai trò của các giai đoạn trong pipeline

Giai đoạn	Nội dung	Bằng chứng không build lại lúc deploy
Chuẩn bị	Đăng nhập GHCR bằng credential, cài QEMU, tạo builder buildx	Token không xuất hiện trong log (--password-stdin)
1. BUILD	Build 3 image bản địa (amd64) --load vào Docker local; đẩy cache :buildcache lên registry	Lần chạy sau được lợi cache, build nhanh hơn
2. TEST	Chạy stack smoke test bằng 3 image vừa build: 10 phép thử phủ frontend, API, proxy và CRUD	10/10 PASS trước khi push
3. PUSH	Buildx đa kiến trúc linux/amd64 và linux/arm64 rồi đẩy 3 image kèm tag :latest	Kiểm tra bằng docker buildx imagetools thấy đủ 2 nền tảng

4. PULL	Xóa image bản địa rồi docker pull thật từ registry	Ép đúng thao tác mà server sẽ làm
5. DEPLOY	docker compose -p student-mgmt-prod -f docker-compose.yml -f docker-compose.prod.yml up -d	Dùng image:, không có build:; giữ volume sqldata
6. Kiểm tra	Gọi backend và frontend qua host.docker.internal, chờ app sẵn sàng (tối đa 270s)	In ra JSON của API và mã HTTP 200

B. Jenkinsfile minh họa

Dưới đây là đoạn mã rút gọn của tệp Jenkinsfile (bản đầy đủ nằm tại thư mục gốc của kho mã nguồn); các lệnh chi tiết trong từng giai đoạn được viết bằng shell sh vì Jenkins chạy trong container Linux:

```
pipeline {
  agent any
  options { timestamps(); timeout(time: 45, unit: 'MINUTES') }
  environment {
    IMAGE_NAME = 'technology-assassin/student-management'
  }
  stages {
    stage('Chuẩn bị') {
      steps { sh 'echo $GHCR_TOKEN | docker login ghcr.io -u $GHCR_USER
--password-stdin' }
    }
    stage('1. BUILD')   { steps { sh 'docker build -t $IMAGE_NAME-
backend:${GIT_COMMIT} backend' } }
    stage('2. TEST')    { steps { sh 'SMOKE_IMAGE_TAG=${GIT_COMMIT}
./scripts/smoke-test.sh' } }
    stage('3. PUSH')    { steps { sh 'docker buildx build --platform
linux/amd64,linux/arm64 --push ...' } }
    stage('4. PULL')    { steps { sh 'docker rmi ...; docker pull
$IMAGE_NAME-backend:${GIT_COMMIT}' } }
    stage('5. DEPLOY')  { steps { sh 'docker compose -p student-mgmt-
prod -f docker-compose.yml -f docker-compose.prod.yml up -d' } }
    stage('6. Kiểm tra'){ steps { sh 'curl -sf
http://host.docker.internal:8080/api/students' } }
  }
  post { always { sh 'docker logout ghcr.io' } }
}
```

Một số điểm kỹ thuật đáng lưu ý của pipeline:

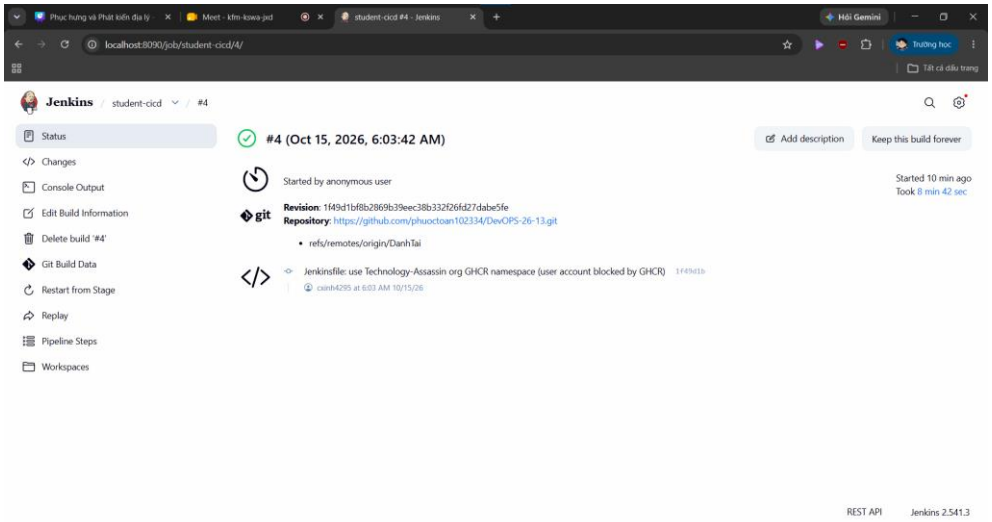
- Smoke test gồm 10 phép thử: trang chủ HTTP 200 và là HTML thật; API /api/students trả về mảng JSON; nginx proxy chuyển /api/ về backend; tạo, tìm, cập nhật và xóa sinh viên qua POST/GET; sau khi xóa, GET trả về danh sách rỗng.

Stack test dùng cổng riêng (3999, 55999, 15333) và tên project theo BUILD_NUMBER nên không đụng stack đang chạy.

- Đa kiến trúc: cài QEMU, dùng buildx để build bản arm64 chạy dưới giả lập; bản amd64 được build native để test nhanh trước khi đẩy.
- Không build lại lúc deploy: docker-compose.prod.yml dùng cú pháp !reset/!override (yêu cầu Docker Compose ≥ 2.24) để thay thế khối build: của compose gốc bằng image: trỏ đúng tag theo commit.
- Rollback: chỉ cần chọn lại tag image của bản trước rồi chạy lại lệnh compose up -d — không cần build lại.

C. Kết quả chạy pipeline

Hình 4.6 là trang kết quả của lần chạy Build #4 thuộc job student-cicd. Dấu tích xanh cho biết toàn bộ sáu giai đoạn hoàn tất thành công; Jenkins lấy mã nguồn từ nhánh DanhTai, build image, chạy smoke test đạt 10/10, đẩy image đa kiến trúc lên GHCR, kéo image về và triển khai; ứng dụng thật sẵn sàng chỉ sau khoảng 9 giây chờ cuối. Bảng 4.6 tổng hợp thông tin chi tiết của build này.



Hình 4.6. Kết quả Build #4 của job student-cicd trên Jenkins

Bảng 4.6. Thông tin Build #4

Mục	Giá trị
Job / Build	student-cicd / #4
Thời điểm bắt đầu	13:03, ngày 15/10/2026
Thời gian chạy	8 phút 42 giây
Trạng thái	Thành công (dấu tích xanh)
Người kích hoạt	Started by user anonymous — kích hoạt qua REST API của Jenkins
Kho mã nguồn	https://github.com/phuoctoan102334/DevOPS-26-13.git

Nhánh	refs/remotes/origin/DanhTai (job khai báo */DanhTai)
Revision (commit)	1f49d1b — “Jenkinsfile: use Technology-Assassin org GHCR namespace”
Tác giả commit	Danh Tài (cxinh4295@gmail.com)
Smoke test	10/10 PASS
Images đã đẩy	ghcr.io/technology-assassin/student-management-{backend,frontend,database}:1f49d1b — linux/amd64 và linux/arm64
Ứng dụng sau deploy	http://localhost:3000 (giao diện), http://localhost:8080/api/students (REST API)
Phiên bản Jenkins	2.541.3 (container jenkins, cổng 8090)

Kết luận: đã xây dựng và chạy thành công pipeline Jenkins sáu giai đoạn cho dự án; mã nguồn được lấy tự động từ GitHub, smoke test đạt kết quả đầy đủ, image đa kiến trúc được đẩy lên registry và ứng dụng được triển khai bằng đúng image đã pull.

CHƯƠNG 5. ĐÁNH GIÁ VÀ KẾT LUẬN

5.1. Kết quả đạt được

Bảng 5.1. Tổng hợp kết quả thực hành

Nội dung	Kết quả	Minh chứng
Tạo docker-compose.yml và chạy lần đầu	Thành công — container chạy, cổng 8000 mở	Hình 4.1
Kiểm tra container, push image và build lại	Nhận diện tag đã tồn tại; build lại bằng --build	Hình 4.2
Hoàn tất push bản mới	Các layer được đẩy, digest mới ghi nhận	Hình 4.3
Kiểm tra ứng dụng web	localhost:8000 hiển thị trang đúng	Hình 4.4
Quản lý container	Container hiển thị Running trên Docker Desktop	Hình 4.5
Jenkins cho dự án	Build #4 thành công sau 8 phút 42 giây; smoke 10/10; image đa kiến trúc; deploy bằng image đã pull	Hình 4.6

5.2. Một số lưu ý và cách xử lý

- **Kho image từ chối push:** lúc đầu image được push vào namespace theo tên tài khoản cá nhân nhưng GHCR trả về lỗi không tìm thấy chủ namespace. Sau khi kiểm tra quyền và thử bằng lệnh push trực tiếp, đã chuyển sang namespace organization mà tài khoản có quyền admin; pipeline chỉ cần sửa một biến IMAGE_NAME là chạy thông. Bài học: kiểm tra quyền và namespace của registry ngay từ đầu.
- **Tag đã tồn tại trên registry:** với Docker Hub, push tag latest mà image chưa có thay đổi sẽ báo đã tồn tại; cần build lại image mới rồi push để có digest mới.
- **Container thoát ngay sau khi chạy:** xảy ra khi không có tiến trình nào giữ container sống; dùng cặp tham số -d và -it, hoặc với Compose để service chạy nền là đủ.
- **Thứ tự khởi động giữa các service:** với bộ ba backend — frontend — database, backend cần chờ database khỏe mạnh (healthcheck) trước khi khởi động; Compose hỗ trợ depends_on kèm điều kiện service_healthy.
- **Trùng cổng và xung đột khi chạy nhiều build:** Jenkins được gán cổng 8090 để tránh trùng backend 8080; smoke test dùng dãy cổng riêng. Khi vô tình kích hoạt hai build cùng lúc, cần hủy build cũ để tránh đua cổng.

- **Bảo mật:** token được lưu trong Jenkins Credentials và chỉ giải nén khi chạy; lệnh docker logout được đặt trong khối post để dọn credential sau mỗi lần build. Dòng "Started by user anonymous" cho thấy Jenkins đang cho phép chạy ẩn danh — chấp nhận được trên máy cá nhân, môi trường thật cần bật xác thực và phân quyền.

5.3. Hướng phát triển

6. Chuyển từ Poll SCM sang GitHub Webhook để build chạy ngay khi có commit.
7. Bổ sung bước kiểm thử chất lượng (unit test, kiểm tra mã) và báo cáo ngay trên Jenkins; cân nhắc tích hợp SonarQube.
8. Ký image và quản lý SBOM để tăng độ tin cậy của artifact khi triển khai.
9. Bật xác thực và phân quyền cho Jenkins, định kỳ xoay vòng token cá nhân (PAT).
10. Khi namespace tổ chức/khác ổn định, có thể đổi IMAGE_NAME về namespace mong muốn chỉ bằng một dòng.
11. Mở rộng sang triển khai liên tục (CD) lên máy chủ hoặc cụm Kubernetes; thêm thông báo kết quả build qua email hoặc ứng dụng nhắn tin cho nhóm.

5.4. Kết luận

Qua bài thực hành, một chuỗi công cụ DevOps cơ bản đã được dựng trên máy Windows cá nhân: Docker với docker-compose để mô tả và chạy ứng dụng, Docker Hub để lưu trữ và phân phối image, và Jenkins để tự động hóa toàn bộ quy trình từ commit trên GitHub đến khi ứng dụng chạy thật. Kết quả Build #4 thành công với smoke test 10/10 và image đa kiến trúc cho thấy quy trình tích hợp — triển khai liên tục đã hoạt động đúng như thiết kế.

Quá trình thực hành giúp hiểu rõ vai trò của từng công cụ và cách chúng ghép lại thành một quy trình mượt mà: code xong chỉ cần git push, mọi việc còn lại là của máy. Những khó khăn gặp phải — từ việc chọn namespace registry phù hợp, quản lý tag image, đến việc sắp xếp thứ tự khởi động giữa các service — đều là kinh nghiệm quý cho các bài thực hành nâng cao hơn về quản lý cấu hình và điều phối container.

TÀI LIỆU THAM KHẢO

[1] Microsoft, Windows Subsystem for Linux Documentation, https://learn.microsoft.com/windows/wsl/

[2] OpenSSH Project, OpenSSH Manual Pages, https://www.openssh.com/manual.html

[3] Docker Inc., Docker Documentation, https://docs.docker.com/

[4] Docker Inc., Docker Compose Documentation, https://docs.docker.com/compose/

[5] Docker Inc., Building with Buildx (multi-platform images), https://docs.docker.com/buildx/working-with-buildx/

[6] Jenkins Project, Pipeline Syntax, https://www.jenkins.io/doc/book/pipeline/syntax/

[7] GitHub, Working with the Container registry (GHCR), https://docs.github.com/en/packages/working-with-a-github-packages-registry/working-with-the-container-registry